

Docker and Kubernetes



Lesson Objectives

Session 1 – Getting started with Docker

Session 2 – Building Docker images

Session 3 – Docker Networking

Session 4 – Docker storage

Session 5 – Making a real project with Docker and NodeJs

Session 6 – Docker Swarm

Session 7-10 – Kubernetes

Session 11 – Final test

Session 2

Building Docker images

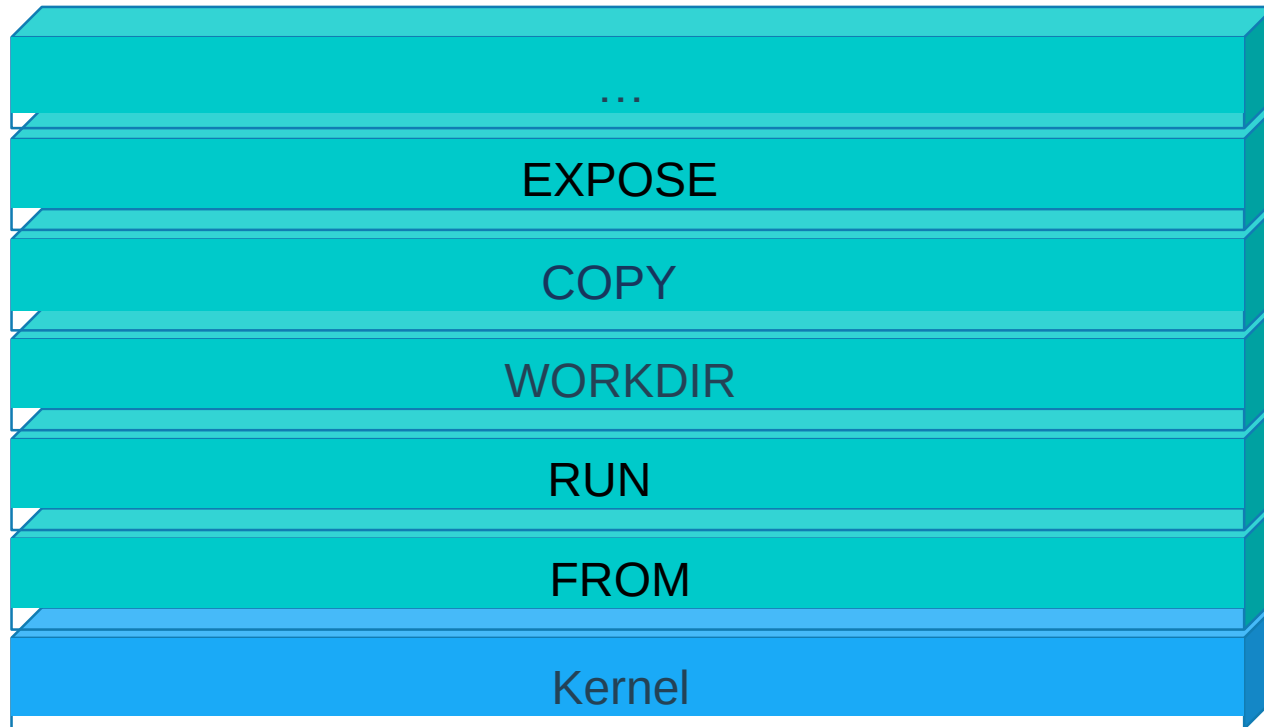
- Dockerfile
- Build parameter
- Environment variables
- CMD and ENTRYPOINT
- Practice building images

Dockerfile – Linux Example

```
Dockerfile x
1  # Create image based on the official Node 6 image from dockerhub
2  FROM node:latest
3
4  # Create a directory where our app will be placed
5  RUN mkdir -p /usr/src/app
6
7  # Change directory so that our commands run inside this new directory
8  WORKDIR /usr/src/app
9
10 # Copy dependency definitions
11 COPY package.json /usr/src/app
12
13 # Install dependencies
14 RUN npm install
15
16 # Get all the code needed to run the app
17 COPY . /usr/src/app
18
19 # Expose the port the app runs in
20 EXPOSE 4200
21
22 # Serve the app
23 CMD ["npm", "start"]
```

- Instructions on how to build a Docker image
- Looks very similar to “native” commands
- Important to optimize your Dockerfile

Each Dockerfile Command Creates a Layer



Docker Image Pull: Pulls Layers

```
Alexander@DESKTOP-90ATKET MINGW64 ~/Docker/Demo
$ docker pull nginx:latest
latest: Pulling from library/nginx
bc95e04b23c0: Pull complete
f3186e650f4e: Pull complete
9ac7d6621708: Pull complete
Digest: sha256:b81f317384d7388708a498555c28a7cce778a8f291d90021208b3eba3fe74887
Status: Downloaded newer image for nginx:latest
```

Here is the format of the Dockerfile:

```
# Comment  
INSTRUCTION arguments
```

The instruction is not case-sensitive. However, convention is for them to be UPPERCASE to distinguish them from arguments more easily.

Docker runs instructions in a Dockerfile in order. A Dockerfile must begin with a FROM instruction. This may be after parser directives, comments, and globally scoped ARGs

Here is the format of the Dockerfile:

```
# Comment  
INSTRUCTION arguments
```

The instruction is not case-sensitive. However, convention is for them to be UPPERCASE to distinguish them from arguments more easily.

Docker runs instructions in a Dockerfile in order. A Dockerfile must begin with a FROM instruction. This may be after parser directives, comments, and globally scoped ARGs

The **FROM** instruction initializes a new build stage and sets the Base Image for subsequent instructions. As such, a valid Dockerfile must start with a **FROM** instruction. The image can be any valid image – it is especially easy to start by pulling an image from the Public Repositories.

FROM can appear multiple times within a single Dockerfile to create multiple images or use one build stage as a dependency for another. Simply make a note of the last image ID output by the commit before each new **FROM** instruction. Each **FROM** instruction clears any state created by previous instructions.

Understand how ARG and FROM interact

FROM instructions support variables that are declared by any ARG instructions that occur before the first **FROM**.

```
ARG CODE_VERSION=latest
FROM
base:${CODE_VERSION}
CMD /code/run-app
FROM
extras:${CODE_VERSION}
CMD /code/run-extras
```

An **ARG** declared before a **FROM** is outside of a build stage, so it can't be used in any instruction after a **FROM**. To use the default value of an **ARG** declared before the first **FROM** use an **ARG** instruction without a value inside of a build stage:

```
ARG VERSION=latest
FROM busybox:$VERSION
ARG VERSION
RUN $VERSION >
image_version
```

RUN has 2 forms:

- ❖ **RUN** <command> (shell form, the command is run in a shell, which by default is /bin/sh -c on Linux or cmd /S /C on Windows)
- ❖ **RUN** ["executable", "param1", "param2"] (exec form)

The default shell for the shell form can be changed using the SHELL command.

```
RUN ["/bin/bash", "-c", "echo  
hello"]
```

```
WORKDIR /path/to/workdir
```

The **WORKDIR** instruction sets the working directory for any **RUN**, **CMD**, **ENTRYPOINT**, **COPY** and **ADD** instructions that follow it in the Dockerfile. If the **WORKDIR** doesn't exist, it will be created even if it's not used in any subsequent Dockerfile instruction.

The **WORKDIR** instruction can be used multiple times in a Dockerfile. If a relative path is provided, it will be relative to the path of the previous **WORKDIR** instruction

```
WORKDIR /a
```

```
WORKDIR b
```

```
WORKDIR c
```

```
RUN pwd
```

COPY has two forms:

```
COPY [--chown=<user>:<group>] <src>... <dest>  
COPY [--chown=<user>:<group>] ["<src>",... "<dest>"]
```

The COPY instruction copies new files or directories from <src> and adds them to the filesystem of the container at the path <dest>.

Multiple <src> resources may be specified but the paths of files and directories will be interpreted as relative to the source of the context of the build.

The **EXPOSE** instruction informs Docker that the container listens on the specified network ports at runtime. You can specify whether the port listens on TCP or UDP, and the default is TCP if the protocol is not specified.

By default, **EXPOSE** assumes TCP. You can also specify UDP:

```
EXPOSE 80/udp
```

The **ENV** instruction sets the environment variable <key> to the value <value>.

The value will be interpreted for other environment variables, so quote characters will be removed if they are not escaped. Like command line parsing, quotes and backslashes can be used to include spaces within values

```
ENV MY_NAME="John Doe"  
ENV MY_DOG=Rex\ The\ Dog  
ENV MY_CAT=fluffy
```


The **CMD** instruction has three forms:

- ❖ `CMD ["executable","param1","param2"]` (exec form, this is the preferred form)
- ❖ `CMD ["param1","param2"]` (as default parameters to ENTRYPOINT)
- ❖ `CMD command param1 param2` (shell form)

An **ENTRYPOINT** allows you to configure a container that will run as an executable.

ENTRYPOINT has two forms:

The exec form, which is the preferred form:

```
ENTRYPOINT ["executable", "param1",  
"param2"]
```

The shell form:

```
ENTRYPOINT command param1  
param2
```

Understand how CMD and ENTRYPOINT interact

The table below shows what command is executed for different **ENTRYPOINT / CMD** combinations:

	No ENTRYPOINT	ENTRYPOINT exec_entry p1_entry	ENTRYPOINT ["exec_entry", "p1_entry"]
No CMD	<i>error, not allowed</i>	/bin/sh -c exec_entry p1_entry	exec_entry p1_entry
CMD ["exec_cmd", "p1_cmd"]	exec_cmd p1_cmd	/bin/sh -c exec_entry p1_entry	exec_entry p1_entry exec_cmd p1_cmd
CMD ["p1_cmd", "p2_cmd"]	p1_cmd p2_cmd	/bin/sh -c exec_entry p1_entry	exec_entry p1_entry p1_cmd p2_cmd
CMD exec_cmd p1_cmd	/bin/sh -c exec_cmd p1_cmd	/bin/sh -c exec_entry p1_entry	exec_entry p1_entry /bin/sh -c exec_cmd p1_cmd

Multi-stage builds

In the Production Environment, Docker image can be so **BIG**, contains many layers

BIG means : Slow, hard to work with, more potential vulnerabilities and possibly a bigger attack surface!

=> Keep images as small as possible

(Tips : every RUN instruction adds a new layer. As a result, it's usually considered a best practice to include

multiple commands as part of a single RUN instruction — all glued together with double-ampersands (&&) and backslash (\) line-breaks)

=> Multi-stage builds comes to rescue!

Multi-stages builds have a single Dockerfile containing multiple FROM instructions.

Each FROM instruction is

a new build stage that can easily COPY artifact from previous stages

Multi-stage builds

Example : Dockerfile built with Multi-stage builds

```
FROM node:latest AS storefront
WORKDIR /usr/src/atsea/app/react-app
COPY react-app .
RUN npm install
RUN npm run build

FROM maven:latest AS appserver
WORKDIR /usr/src/atsea
COPY pom.xml .
RUN mvn -B -f pom.xml -s /usr/share/maven/ref/settings-docker.xml dependency:resolve
COPY . .
RUN mvn -B -s /usr/share/maven/ref/settings-docker.xml package -DskipTests

FROM java:8-jdk-alpine AS production
RUN adduser -Dh /home/gordon gordon
WORKDIR /static
COPY --from=storefront /usr/src/atsea/app/react-app/build/ .
WORKDIR /app
COPY --from=appserver /usr/src/atsea/target/AtSea-0.0.1-SNAPSHOT.jar .
ENTRYPOINT ["java", "-jar", "/app/AtSea-0.0.1-SNAPSHOT.jar"]
CMD ["--spring.profiles.active=postgres"]
```

Stage 1: storefront : Lots of build stuff,
not much production code

Stage 2: appserver : Lots of build stuff,
not much production code

Stage 3: production : Contains
Production artifacts built from storefront
Appserver stage

Docker image build -t multi-stage .

...

Docker image ls

REPO	TAG	IMAGE ID	CREATED	SIZE
node	latest	a5a6a9c32877	5 days ago	941MB
<none>	<none>	d2ab20c11203	9 mins ago	1.11GB
maven	latest	45d27d110099	9 days ago	508MB
<none>	<none>	fa26694f57cb	7 mins ago	649MB
java	8-jdk-alpine	3fd9dd82815c	7 months ago	145MB
multi	stage	3dc0d5e6223e	1 min ago	210MB

Leverage Build cache

The build process used by Docker has the concept of a cache that it uses to speed-up the build process : This is because artifacts from the first build, such as layers, are cached and leveraged by later builds

Image build process iterates through a Dockerfile one-line-at-a-time starting from the top. For each instruction, Docker looks to see if it already has an image layer for that instruction in its cache :

If it does, this is a cache hit and it uses that layer.

If it doesn't, this is a cache miss and it builds a new layer from the instruction. Getting cache hits can hugely speed up the build process

You can force the build process to ignore the entire cache by passing the `--no-cache=true` flag to the docker image build command (Use case : to keep getting latest layer from Docker hub)

